Findings: Cellar vs. a Formex-based CDM Graph

Sharanya Sridharan

10 November 2026

Summary

1. The public Cellar SPARQL endpoint has **no instances** of `cdm:article_legal` or of any of the 15 `cdm:subdivision` subclasses, although CDM defines them. Structure below the act is not available from Cellar.
2. CDM can be reused for the structure. Subdivisions are Works, so they inherit the Work properties for number, title and sequence. CDM has **no property for the text** of a provision.
3. A Formex mapping (Regulation 2026/1744) adds articles, recitals, English expressions with headings and text, each with its own URI and a link to the act.
4. The text can be kept as a literal in the graph, or outside it with a pointer (Section 5).

1 Approach

Cellar was queried through its public SPARQL form (sponging off, local data only). IRIs were copied from the CDM ontology pages, and each negative result was paired with a control query that returns rows. The Formex graph was generated with YARRRML/RML (RMLMapper 8.1.0) for Regulation (EU) 2026/1744. Earlier tests used the NIS Directive (32016L1148) and the AI Act notice.

2 SPARQL queries on Cellar and results

All results below are from 5 November 2026.

2.1 Control: is the endpoint usable in local mode?

```
PREFIX cdm: <http://publications.europa.eu/ontology/cdm#>
SELECT ?s ?o WHERE { ?s cdm:expression_belongs_to_work ?o } LIMIT 5
```

Result: 5 rows. The setup works, and Cellar stores this direction as real triples.

2.2 Inverse property pairs

```
PREFIX cdm: <http://publications.europa.eu/ontology/cdm#>
SELECT ?s ?o WHERE { ?s cdm:work_has_expression ?o } LIMIT 5
```

Result: empty. CDM defines many properties in inverse pairs (here `cdm:work_has_expression` and `cdm:expression_belongs_to_work`). Cellar stores one direction and does not materialise the other, so an empty result for one direction says nothing about the other. Both directions must be tested before concluding anything.

2.3 Are there article-level resources?

```
PREFIX cdm: <http://publications.europa.eu/ontology/cdm#>
ASK { ?s a cdm:article_legal }
```

Result: `false` (and the equivalent `SELECT ... LIMIT 10` returns no rows). The "Show property instances" table of `cdm:article_legal_part_of_work` is empty as well.

2.4 Subdivision classes

The class `cdm:subdivision` has 15 subclasses (Section 4). Each of them was checked with the same `ASK` pattern, and **none has an instance** in the public endpoint.

2.5 Targeted lookups (supporting checks)

Point lookups are cheap; full scans over all triples tend to hang on the public endpoint and were avoided.

```
# what does Cellar say about the act?
SELECT ?p ?o WHERE {
  <http://publications.europa.eu/resource/cellar/dc8116a1-3fe6-11ef-865a-01
   aa75ed71a1> ?p ?o
} LIMIT 50

# what points to the NIS directive?
PREFIX cdm: <http://publications.europa.eu/ontology/cdm#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT DISTINCT ?s ?p WHERE {
  VALUES ?c { "32016L1148" "32016L1148"^^xsd:string }
  ?w cdm:resource_legal_id_celex ?c .
  ?s ?p ?w .
} LIMIT 300
```

2.6 Summary of Cellar findings

Question	Result
Endpoint usable (control query)	yes, 5 rows
Instances of `cdm:article_legal`	none (`ASK` false)
Instances of the 15 subdivision subclasses	none
Property for the body text of a provision	none in CDM
Full text available as triples	no; only in files behind manifestations

Scope. These results describe the *public* endpoint on 5 November 2026. A point lookup of the type of the treaty article `eli/treaty/tfeu_2016/art_114/oj` returned no type triple, so the endpoint gives no class for treaty articles either.

3 What the Formex mapping adds

The mapping for Regulation 2026/1744 produces the following entities (counts from the test run).

Entity	Count	Main properties
Act	1	`cdm:work`, `cdm:regulation`, CELEX identifier
Article	4	`cdm:article_legal`, `cdm:article_legal_part_of_work`, `ex:formexIdentifier`, `ex:number`
Article expression (ENG)	4	`cdm:expression`, `cdm:expression_belongs_to_work`, `cdm:expression_uses_language`, `cdm:expression_title`, `ex:text`
Recital	47	`cdm:recital`, `cdm:recital_part_of_work`, `ex:number`, `ex:text`

Only the direct children of `ENACTING.TERMS` are mapped. Regulation 2026/1744 is an amending act, so articles and paragraphs inside `QUOT.S` are quoted text of *other* acts and are deliberately excluded.

3.1 Compared with Cellar

Information	Cellar endpoint	Formex graph
Act-level metadata (dates, relations, classification)	yes	no (only CELEX)
Articles and recitals as resources	no instances	yes
Article number, heading	no	yes
Link from provision to act	no	yes
Body text of a provision	no	yes (`ex:text`)
Addressable URI for one provision	no	yes (own namespace)

The two sources are complementary: Cellar gives rich act-level metadata, the Formex graph gives the structure and the text below the act. Linking them (for example `owl:sameAs` from the act URI to the Cellar resource) combines both.

3.2 Practical points of the mapping

- Provision URIs are derived from the document's own identifiers (`@IDENTIFIER` for articles, a cleaned number for recitals), so a URI points to a specific provision.
- The raw Formex needs a cleaning step (remove footnotes, add spaces between blocks, add a clean number attribute to recitals). The service applies it when the "Clean Formex before mapping" option is on. Without it, words run together and recitals get no readable number. It must stay off for Cellar metadata XML.
- The YARRRML parser cannot handle XPath functions with parentheses inside `$()`, which is why the cleaning is done before the mapping.

4 Reusable CDM classes for subdivisions

`cdm:subdivision` is a subclass of "Superclass for all text structures", which is a subclass of `Work`. Every subdivision is therefore a Work and inherits the Work properties. The Work class also has the property `cdm:work_contains_subdivision`, with one sub-property per subdivision class, each paired with an inverse "... part of work" property.

Class	Part-of-work property	Work contains property
alinea	alinea_part_of_work	work_contains_alinea
annex	annex_part_of_work	work_contains_annex
article_legal	article_legal_part_of_work	work_contains_article_legal
chapter_legal	chapter_legal_part_of_work	work_contains_chapter_legal
citation	citation_part_of_work	work_contains_citation
clause_legal	clause_legal_part_of_work	work_contains_clause_legal
indent	indent_part_of_work	work_contains_indent
paragraph_legal	paragraph_legal_part_of_work	work_contains_paragraph_legal
part_legal	part_legal_part_of_work	work_contains_part_legal
point	point_part_of_work	work_contains_point
preamble	preamble_part_of_work	work_contains_preamble
recital	recital_part_of_work	work_contains_recital
rule_legal	rule_legal_part_of_work	work_contains_rule_legal
section_legal	section_legal_part_of_work	work_contains_section_legal
title_legal	title_legal_part_of_work	work_contains_title_legal

All IRIs are in the namespace `http://publications.europa.eu/ontology/cdm#` (prefix `cdm:`).

Definitions given in the ontology: article, paragraph, part, chapter, citation, clause, rule, title and recital are each "a ... in a legal document". The ontology notes that citation, clause, rule, chapter, title and recital were "created to have correspondence to the subdivision NAL" (the Publications Office authority table of subdivision types). The class `cdm:annex` ("an addition to a document") has a second super-class, "Auxiliary works". There is also an `cdm:appendix` property pair under `cdm:work_contains_subdivision`.

4.1 Mapping of Formex elements

Formex element	CDM class	Status
ARTICLE	cdm:article_legal	mapped
CONSID	cdm:recital	mapped
PREAMBLE	cdm:preamble	not yet mapped
VISA	cdm:citation	not yet mapped
PARAG	cdm:paragraph_legal	not yet mapped
ALINEA	cdm:alinea	not yet mapped
NP, ITEM	cdm:point, cdm:indent	not yet mapped
ANNEX	cdm:annex	not yet mapped
TITLE, DIVISION	cdm:title_legal, cdm:chapter_legal, cdm:section_legal, cdm:part_legal	not yet mapped

4.2 Reusable properties

Need	CDM term
Subdivision to its work	the “part of work” property of its class (table above)
Subdivision to its parent	`cdm:subdivision_part_of_subdivision` (sub-property of `cdm:work_part_of_work`)
Number, title, order	datatype properties defined on the Work class and so inherited by every subdivision: `cdm:work_has_number` (string), `cdm:work_title` (literal, may carry several language versions), `cdm:work_sequence` (positive integer, “order of the work in a group of works”). `cdm:work_id` (“Work identifier”) is marked deprecated in the ontology and is not used
Language version	`cdm:expression`, `cdm:expression_belongs_to_work`, `cdm:expression_uses_language`, `cdm:expression_title`
Source file	Manifestation formex

Not covered by CDM. There is no property for the body text of a provision. CDM provides only titles, subtitles, abstracts and editorial descriptions at expression level, and file-level descriptions at manifestation and item level. The text lives only in the files that items point to. The mapping therefore uses one extension property, `ex:text`, in its own namespace (`http://example.org/vocab#`), with a language-tagged literal.

5 Two ways to store and access the plain text

Because CDM has no text property, the text must live somewhere and be reachable from the provision’s URI.

5.1 Option A: text as a literal in the graph

```
<.../art_1.ENG> cdm:expression_belongs_to_work <.../art_1> ;
                ex:text "Article text ..."@en .
```

Access. A single SPARQL query returns structure and text together:

```
PREFIX cdm: <http://publications.europa.eu/ontology/cdm#>
PREFIX ex:  <http://example.org/vocab#>
SELECT ?n ?text WHERE {
  ?art a cdm:article_legal ; ex:number ?n .
  ?exp cdm:expression_belongs_to_work ?art ; ex:text ?text .
} ORDER BY ?n
```

Keyword filtering with `FILTER(CONTAINS(LCASE(STR(?text)), "word"))` works, but it is a substring match without ranking.

Advantages. One system, one query language, no extra service. Structure and text cannot drift apart. Language tags handle several languages naturally.

Drawbacks. The graph grows with the text size; long literals slow loading and some triple stores handle them badly. No ranked full-text search without a store-specific text index. Duplicates the text that already exists in Cellar’s files.

5.2 Option B: text outside the graph, referenced by pointer

The graph holds only structure and a pointer to where the text is, for example the Formex item and the element in it, or an ID in a separate store (files, a document database, a search or vector index).

```
<.../art_1.ENG> ex:textSource <.../DOC_1> ;      # the Formex item
                ex:textLocator "//ARTICLE[@IDENTIFIER='001']" .
```

Access. Two steps: (1) a SPARQL query finds the provisions and their pointers; (2) the application fetches the text from the file or index using the pointer. A small service function can hide both steps, for example `get_text(uri)`.

Advantages. The graph stays small and fast. The text can sit in a store built for it, such as a full-text or vector index, which suits RAG retrieval and ranking. The pointer ties the text back to the official Formex item in Cellar, which fits the FRBR idea that text belongs to the item and not to the work.

Drawbacks. Two systems to keep in sync. A pointer can break if the file or XPath changes. Queries that combine text conditions with structure need an application layer.

5.3 Comparison and recommendation

Criterion	A: literal in graph	B: pointer to external store
Complexity	low	higher (two systems)
Graph size	grows with text	small
Single-query structure + text	yes	no, two steps
Ranked full-text / vector search	limited	good
Consistency	automatic	must be maintained
Fits the proof of concept	best	later stage

For the proof of concept, Option A is the simpler choice and is what the current mapping implements. If the graph is later used for retrieval over many acts, Option B (or a hybrid: short text in the graph, full text and embeddings in an index, linked by the provision URI) scales better.

6 Open points

- Extend the mapping to paragraphs, points, chapters and annexes, and decide how to represent amendment steps in `QUOT.S`.
- Link the act in the Formex graph to its Cellar resource, for example with `owl:sameAs`.
- Decide between Option A and Option B for storing the text.